

# 9

## Extra laboratory 2

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| <b>9</b> | <b>Human silhouette detection .....</b>       | <b>81</b> |
| 9.1      | Main objectives:                              |           |
| 9.2      | Histogram of Oriented Gradients – description |           |
| 9.3      | Support Vector Machines – description         |           |
| 9.4      | HOG descriptor                                |           |
| 9.5      | SVM classifier                                |           |
| 9.6      | Human silhouette detection                    |           |



## 9. Human silhouette detection

### 9.1 Main objectives:

- familiarisation with the detection of human silhouettes,
- familiarisation with the Histogram of Oriented Gradients descriptor (HOG),
- familiarisation with the Support Vector Machine classifier (SVM),
- basics of machine learning (methodology).

### 9.2 Histogram of Oriented Gradients – description

Histogram of oriented gradients (HOG)<sup>1</sup> is a feature descriptor widely used in image processing and image recognition. It describes the local appearance and shape of an object through the distribution of local intensity gradients and edge directions. The HOG algorithm can be divided into several steps:

1. Gradient calculation,
2. Computation of histogram of gradient orientations in cells,
3. Normalisation of histograms in blocks of cells,
4. Determination of the feature vector.

#### Gradient calculation

For each pixel, a horizontal and vertical gradient is calculated using two one-dimensional masks:  $\begin{pmatrix} -1 & 0 & 1 \end{pmatrix}$  and  $\begin{pmatrix} -1 & 0 & 1 \end{pmatrix}^T$ . As a result, two components of the gradient are obtained, with which its amplitude and orientation can be determined.

For a greyscale image, the amplitude of the gradient is calculated based on the formula (9.1):

$$m(x, y) = \sqrt{G^x(x, y)^2 + G^y(x, y)^2} \quad (9.1)$$

---

<sup>1</sup>Dalal, Navneet, and Bill Triggs. "Histograms of oriented gradients for human detection", 2005 IEEE computer society conference on computer vision and pattern recognition (CVPR'05). Vol. 1. IEEE, 2005.

where:

- $(x, y)$  – pixel coordinates,
- $G^x(x, y)$  – horizontal component of the gradient,
- $G^y(x, y)$  – vertical component of the gradient,
- $m(x, y)$  – gradient amplitude.

When processing a colour image, the gradient amplitude for each channel of the RGB colour space is determined and the highest value selected – equation (9.2).

$$m(x, y) = \max(m_R(x, y), m_G(x, y), m_B(x, y)) \quad (9.2)$$

where:

- $m_R(x, y), m_G(x, y), m_B(x, y)$  – the amplitude of the gradient for each RGB channel.

The orientation of the gradient is calculated from the equation (9.3). When the detector window contains pixels in RGB colour space, in each cell the gradient angle is determined only for the gradient components for which the amplitude is the highest.

$$\theta(x, y) = \arctan\left(\frac{G_{max}^y(x, y)}{G_{max}^x(x, y)}\right) \quad (9.3)$$

where:

- $\theta(x, y)$  – gradient orientation,
- $G_{max}^y(x, y), G_{max}^x(x, y)$  – gradient components for the highest amplitude among the gradients of individual RGB channels.

### Histogram calculation

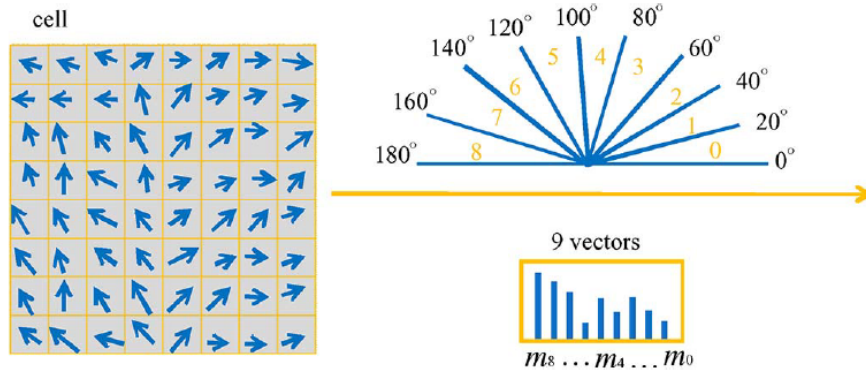


Figure 9.1: View of one cell of  $8 \times 8$  pixels in the detection frame. Distribution of histogram intervals in the range  $0^\circ$  to  $180^\circ$  and an example histogram.

The detector window is divided into non-overlapping cells. The most commonly used cell size is  $8 \times 8$  pixels due to easy hardware implementation. For each cell, a histogram consisting of evenly spaced intervals is determined, representing gradient orientations ranging from  $0^\circ$  to  $180^\circ$  or  $0^\circ$  to  $360^\circ$ . In Figure 9.1 an example of the histogram most commonly used in pedestrian detection is given. In the histogram, the amplitude values of the gradients are accumulated on the basis of the direction in individual intervals. The adjacency of

an individual gradient is accurately determined by the equation (9.3). To compensate for aliasing, i.e. different assignments of gradient values to two adjacent intervals with a small change in gradient orientation, linear interpolation is used. Consequently, the amplitude of the gradient is linearly assigned to adjacent intervals according to the formulas (9.4) and (9.5).

$$v_{UB}(x,y) = m(x,y) \left( \frac{\theta(x,y) - \theta_{LB}}{BW} \right) \quad (9.4)$$

$$v_{LB}(x,y) = m(x,y) \left( \frac{\theta_{UB} - \theta(x,y)}{BW} \right) \quad (9.5)$$

where:

$v_{UB}(x,y), v_{LB}(x,y)$  – amplitude value coming into one of two adjacent predefined intervals of the histogram,

$BW$  – the width of the histogram interval,

$m(x,y)$  – gradient amplitude,

$\theta(x,y)$  – gradient orientation,

$\theta_{LB}$  – the centre of the lower bin of the histogram,

$\theta_{UB}$  – the centre of the upper bin of the histogram.

### Normalisation of histograms and determination of the feature vector

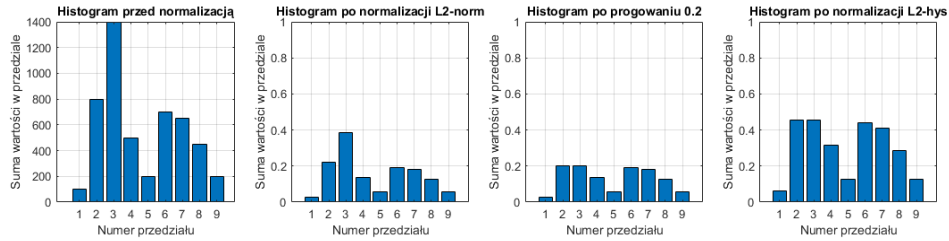


Figure 9.2: An example of a histogram normalisation procedure.

The norm of the gradient within the detection window can vary significantly. For this reason, local normalisation of histograms is introduced. Each block consists of four contiguous cells in the form  $2 \times 2$ . The areas of consecutive blocks overlap by 50% in both directions (vertical and horizontal). This ensures that edges close to the edges of the cells are given the same consideration as those in the centre. The resulting histograms are grouped into a 36-element vector within one block. To reduce the influence of uneven illumination in the detection frame, double normalisation was applied: L2 (formula (9.6)) and L2-hys (formula (9.7)). In Figure 9.2 an example is shown.

In the first stage, the feature vector is reduced to a value between  $[0;1]$ . In this way, the various contrasts present in the image are eliminated. Additional noise may appear on the image in the form of variable illumination caused by the mapping of colour intensity by the camera and reflections from various types of surfaces. As a result, the amplitudes of some parts of the image are significantly higher than in other parts. To reduce the influence of these areas on the shape of the whole histogram, upper thresholding is used. Vector values above 0.2 are truncated and aligned to this number. The vector is then

re-normalised to the interval  $[0;1]$ . As a result, the orientation of the gradient is more influential than its amplitude. The value of the threshold was determined experimentally in D. Lowe's work<sup>2</sup>.

- norm L2:

$$f_1 = \frac{f}{\sqrt{\|f\|_2^2 + \epsilon^2}} \quad (9.6)$$

- norm L2-hys:

$$f_2 = \frac{f_{th}}{\sqrt{\|f_{th}\|_2^2 + \epsilon^2}}, f_{th} = \min(f_1, 0.2) \quad (9.7)$$

where:

- $f$  – unnormalised feature vector,
- $f_1$  – normalised feature vector (norm L2),
- $f_2$  – normalised feature vector (norm L2-hys),
- $f_{th}$  – feature vector after thresholding,
- $\|f\|_k$  – k-th norm of the vector  $f$ ,
- $\epsilon$  – an appropriate constant (avoidance of division by 0).

Finally, the vectors created in each block are concatenated into a single one-dimensional vector, which reaches a size of  $7 \times 15 \times 36 = 3780$  values. This is then passed to the classifier to assign the object to the appropriate class.

### 9.3 Support Vector Machines – description

The aim of the SVM algorithm is to find a (hyper)plane in a multidimensional space defined by features, derived from a descriptor, which is maximally distant from all points of both classes. It is described by the following formula (9.8) and illustrated by the example in Figure 9.3.

$$f(x) = w^T x + b = 0 \quad (9.8)$$

where:

- $x$  – feature vector,
- $w$  – vector of weights, hyperplane parameters,
- $b$  – bias.

There are two versions of the SVM algorithm:

- *soft-margin* – a classifier with a soft margin that allows misclassification of part of the data,
- *hard-margin* – a classifier that unambiguously separates the data from the two classes.

---

<sup>2</sup>David G. Lowe. „Distinctive image features from scale-invariant keypoints”. In: International journal of computer vision 60.2 (2004), s. 91–110.

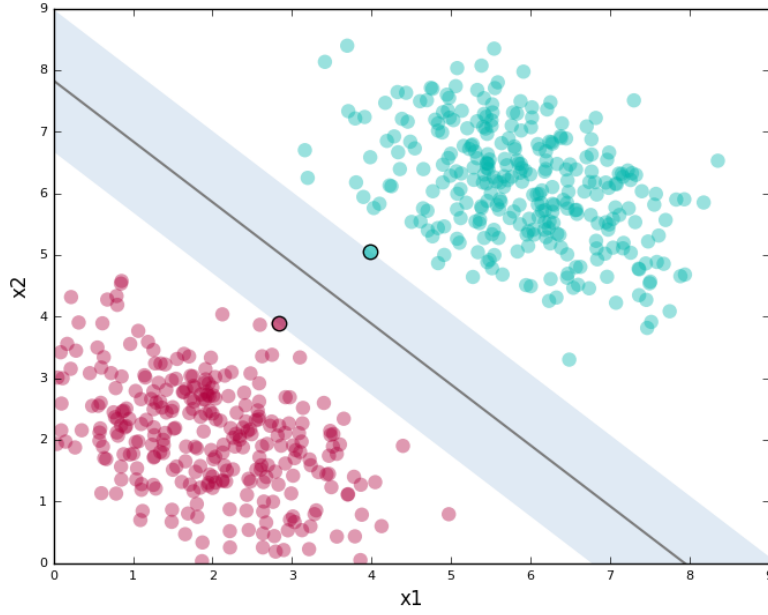


Figure 9.3: Representation of a one-dimensional hyperplane separating samples of two classes for a hard margin classifier.

However, due to the appearing computational inaccuracies and deformations in the image, feature sets are usually not linearly separable. For this reason, a penalty function with a constant coefficient  $C$  is introduced into the optimisation task, which allows a certain group of points to be misclassified. This makes it possible to find a plane that separates the significantly different features of an object, and the remaining points, close to the separation boundary, will not be affected. The introduction of a weakened classifier or in other words a soft margin classifier has an additional advantage. This algorithm is not sensitive to data far from the point of densest clustering and it follows that it generalises the problem better and is resistant to over-fitting. The important issue is to find such a coefficient  $C$  to obtain the lowest percentage of misclassified test samples. Most often cross-validation is used to determine the best value. The larger the parameter, the narrower the margin of the classifier. Figure 9.4 shows the dependence of the value of  $C$  on the width of the classifier margin.

By solving the equation (9.9), a hyperplane is obtained that best generalises the pedestrian classification problem. The set of detection windows, derived from a single image frame, is expressed by the set of pairs:  $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ , where  $x_i$  is the feature vector describing the detection window and  $y_i \in \{-1, 1\}$  are class labels (non-pedestrian, pedestrian).

$$\min_{w, b, \xi} \left( \frac{1}{2} w^T w + C \sum_j \xi_j \right) \quad (9.9)$$

on the assumption that:

$$(w^T x_i + b) y_i \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad (i = 1, \dots, n) \quad (9.10)$$

where:

$w$  – vector of weights, hyperplane parameters,

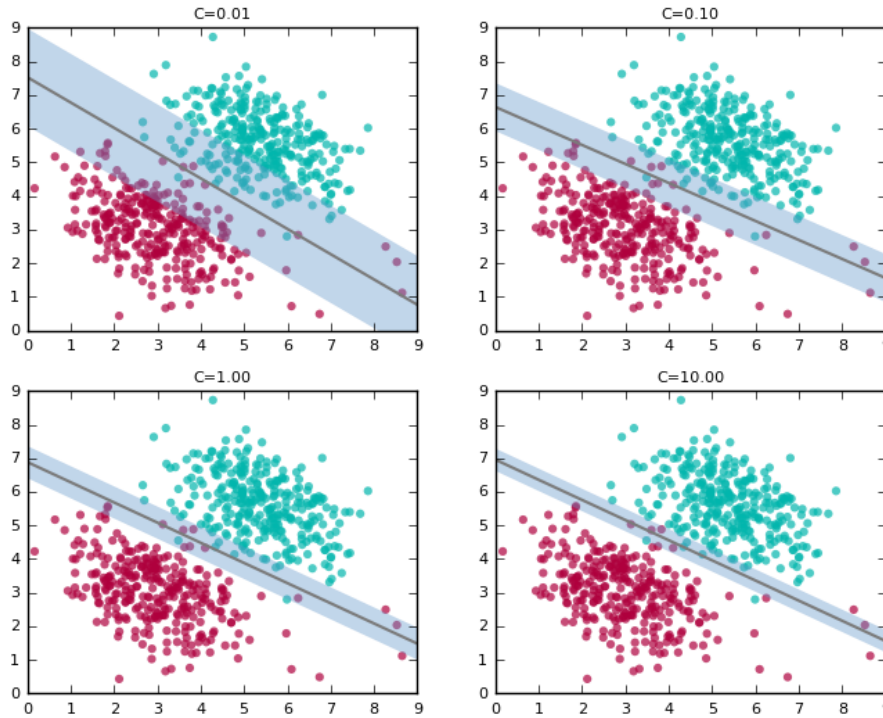


Figure 9.4: Dependence of the parameter  $C$  as a function of penalty on margin width.

$C$  – a constant factor for the penalty function,

$\xi_i$  – slack variable, distance of a point  $(x_i, y_i)$  from the margin on the corresponding side of the hyperplane.

The parameter  $\xi_i$  reduces the separation margin of the samples. There are three intervals that define the position of a given sample relative to the hyperplane. If the condition  $0 \leq \xi_i < 1$  is true, then the point  $(x_i, y_i)$  lies on the correct side of the hyperplane and the classification will be correct. On the other hand, if  $1 < \xi_i$  is true, then point  $(x_i, y_i)$  lies on the wrong side, so the class membership will be incorrectly determined. If the value of  $\xi_i$  equals 1 then it is not possible to correctly assign a label to the sample.

**R** It is worth mentioning again that the above operations are performed iteratively for each image frame ( $64 \times 128$  px) using the *sliding window* method. The list of all parameters of the *HOG+SVM* algorithm is as follows:

- bins* – the number of bins in the histogram (usually 9, in  $20^\circ$  steps),
- cell* – the number of pixels in the cell (usually  $8 \times 8$  px),
- block* – number of cells in the block (usually  $2 \times 2$  cells),
- overlap* – the number of cells common to two neighbouring blocks or the percentage of overlap between blocks (2 cells, 50%),
- frame\_width* – detection frame size (usually  $64 \times 128$  px),
- svm\_weights* – the number of SVM classifier weights (3780 for  $64 \times 128$  px),
- bias\_weight* – bias value of SVM classifier,
- stride* – number of pixels per which the detection frame is moved (usually 8 or its multiple for the frame  $64 \times 128$  px),



## 9.4 HOG descriptor

In a very simple way, the algorithm for feature detection can be divided into three steps:

- calculation of gradients,
- determination of histograms of gradients,
- normalisation of histograms.

At the beginning, please pay attention to the parameters of the algorithm:

- colour space – RGB,
- no gamma correction (or other preprocessing),
- form of gradient operator,
- number of histogram bins, orientations analysed,
- cell size,
- norm type – instead of *norm L2-hys*, we will use *norm L2*,
- block size, cell overlap – stride,
- detection window size.

**Exercise 9.1** The aim of the exercise is to implement the HOG descriptor.

1. From the course page, download a set of image snippets with silhouettes of people (positive samples) and without people (negative samples).
2. Load any positive image – we will develop a HOG implementation for it. You can implement the above three steps in separate functions.
3. We start by calculating the gradient. Vertical and horizontal edge detection should be realised and the amplitude and orientation (in degrees) in each RGB channel should be counted separately. It is best to adapt the function to both greyscale and RGB images. A function can be used to calculate gradients with a mask (from the 9.2 section):

```
dx = scipy.ndimage.filters.convolve1d(np.int32(im), np.array([-1, 0, 1]),1)
dy = scipy.ndimage.filters.convolve1d(np.int32(im), np.array([-1, 0, 1]),0)
```

After calculating the gradients for the three components, select the one with the largest value from them. This can be done in a loop, but it will be inefficient. It is better to use Python's capabilities where values that fulfil a condition are selected from an array. For example, use the function `np.logical_and()`:

```
max_B = np.logical_and(magnitude[:, :, 1] < magnitude[:, :, 0], magnitude
[:, :, 2] < magnitude[:, :, 0])
```

We then analogously find the highest gradients from the remaining components. By selecting the highest gradients from the individual components, we obtain an array containing the maximum values of the gradients. The same mechanism should be used to determine the orientation matrix of the gradients.

4. The second stage – determining histograms in blocks is the most difficult part of the implementation (technically, because conceptually it is quite simple). First, you need to define the cell size (8) and determine the number of cells in the X and Y axis (e.g. `YY_cell = np.int32(YY/cellSize)`). We then create a container for the histograms – one histogram for each cell. We loop through the cells and perform the following operations:
  - We retrieve the matrix values with amplitude and orientation for the corresponding cell.
  - Firstly, you need to handle the case of negative angles (by adding 180 if you

are considering values in degrees).

- Secondly, we need to determine to which interval of the histogram the angle belongs. For this purpose, the interval  $[0;180]$  is divided into 9 parts, and for each of them the middle element should be determined. Then we need to determine to which interval each gradient belongs and to which part of the interval, e.g. for the interval  $0-20^\circ$ , the middle element is 10. So the gradient with orientation  $11^\circ$  belongs to the interval  $0-20^\circ$  and the subinterval  $10-20^\circ$ . This is important because of the necessary use of bilinear interpolation between two adjacent intervals. **Please note** the special treatment of the transition  $180^\circ-0^\circ$  (wrapping).
- The correct result at this stage is to get the corresponding interval number (ideally assumed to be the lower interval). Another solution is to store the compartment numbers for the lower and upper interval.
- Then we need to calculate the distance from the angle to the centre of the intervals according to the formulas (9.4) and (9.5). Here again, remember to handle values close to  $180^\circ$  in a special way.
- Finally, we add the amplitude values from the cell to the corresponding intervals of the histogram.

The final step is to normalise the cells in the blocks. Loop through the cell coordinates (size minus 1) to retrieve 4 histograms belonging to one block. These should then be concatenated into a single vector (`np.concatenate`), the norm calculated (function `np.linalg.norm`) and normalised using L2. The normalised vector is part of the final feature vector.

```
# Normalisation in block
e = math.pow(0.00001,2)
F = []
for jj in range(0,YY_cell-1):
    for ii in range(0,XX_cell-1):
        H0 = hist[jj,ii,:]
        H1 = hist[jj,ii+1,:]
        H2 = hist[jj+1,ii,:]
        H3 = hist[jj+1,ii+1,:]
        H = np.concatenate((H0, H1, H2, H3))
        n = np.linalg.norm(H)
        Hn = H/np.sqrt(math.pow(n,2)+e)
        F = np.concatenate((F,Hn))
```

In the end we should get a vector of 3780 numbers. The first 10 values for the image *per00060.ppm*:

```
0.34384495
0.15958051
0.10338961
0.13061814
0.13671541
0.09076598
0.09694051
0.19526622
0.19388454
0.16014419
```

You can also use the code below to display the histogram gradients (before normal-

ising them).

```
def HOGpicture(w, bs): # w - histograms, bs - cell size (8)
    bim1 = np.zeros((bs, bs))
    bim1[np.round(bs//2):np.round(bs//2)+1,:]= 1;
    bim = np.zeros(bim1.shape+(9,));
    bim[:, :, 0] = bim1;
    for i in range(0,9): #2:9,
        bim[:, :, i] = scipy.misc.imrotate(bim1, -i*20, 'nearest')/255

    Y,X,Z = w.shape
    w[w < 0] = 0;
    im = np.zeros((bs*Y, bs*X));
    for i in range(Y):
        iisl = (i)*bs
        iisu = (i+1)*bs
        for j in range(X):
            jjsl = j*bs
            jjsu=(j+1)*bs
            for k in range(9):
                im[iisl:iisu,jjsl:jjsu] += bim[:, :, k] * w[i,j,k];
    return im
```

Finally, please convert the created code (convenient at the implementation stage) into a function, whose argument is the image (matrix), and return the determined vector of features (normalised gradient histograms).

## 9.5 SVM classifier

The determined feature vectors using the HOG descriptor should then be classified accordingly in order to assess whether or not there is an object (pedestrian) in the analysed image frame.

**Exercise 9.2** The aim of this exercise is to implement an SVM classifier and its training.

1. We start by preparing the dataset. From the course page, download the **pedestrians** folder. It contains positive (with pedestrians) and negative (without pedestrians) samples.
2. For the images in both sets, we calculate the HOG descriptor (for the first 100).

```
HOG_data = np.zeros([2*100,3781],np.float32)

for i in range(0,100):
    IP = cv2.imread('pos/per%05d.ppm' % (i+1))
    IN = cv2.imread('neg/neg%05d.png' % (i+1))
    F = hog(IP)
    HOG_data[i,0] = 1;
    HOG_data[i,1:] = F;
    F = hog(IN)
    HOG_data[i+100,0] = 0;
    HOG_data[i+100,1:] = F;
```

The data contains a class index (1 – pedestrian, 0 – non pedestrian) in addition to the vector.

3. Before training, we separate the input data into a vector of labels (first column) and data.

```
labels = HOG_data[:,0];
data = HOG_data[:,1:]
```

- Include the module from `sklearn` import `svm`.  
 Create a linear SVM object: `clf = svm.SVC(kernel='linear', C = 1.0)`.  
 Carry out training: `clf.fit(data, labels)`.  
 And test: `lp = clf.predict(data)`.
4. We analyse the results of training. Classification is performed on the training set. We determine the confusion matrix by counting the coefficients TP, TN, FP, FN. The results of good training should be close to 100 %.
  5. We carry out a validation of the solution. We basically repeat the above experiment. This is the moment to make any corrections:
    - change of parameter C,
    - change the kernel, etc.
 Of course, after making changes, you should always check their effect on the test and validation sets.
  6. Finally, once we are sure that a particular version of the classifier achieves the best results, we run a check on the test set. We consider its results as final for the classifier. This is at least methodologically correct.

## 9.6 Human silhouette detection

At this point, we already have all the tools to implement a complete detection system: the HOG descriptor and the trained SVM classifier. We can therefore implement the detection on the real image.

**Exercise 9.3** Pedestrian silhouette detection using a HOG descriptor and a trained SVM classifier.

1. Four test images are provided on the course page: (`testImage1-4.png`). But you can just as well use any images you like.
2. At first, it is worth checking if the height of the person in the image roughly corresponds to the size of our detection window. This can be done manually (measure the height in pixels) or automatically (draw the detection window on the image). The second method is better as we will still need the rectangle drawing function on the image (to visualise the detection). As a reminder `cv2.rectangle`.
3. Next we need to adjust the size of the image (function `cv2.resize`) to the height of the figure. It is worth browsing for ourselves to see what the silhouettes look like in the training set – this will give us an idea of “how the classifier might have learned”.
4. We implement a loop over the image. In each iteration we take a patch of size  $64 \times 128$ , calculate the HOG descriptor for it, and then do the classification. If it is positive, then we apply a rectangle to the image that is copy to be displayed (we make a copy using the copy method).

**R** The step (i.e. how much we move the window vertically and horizontally) should not be 1, because we may not get the result. It is best to choose it empirically, but values of 8 or 16 seem appropriate.

5. Try if the detection works on the attached images.
6. **Optional.** Multi-scale processing.  
 Previously we selected the appropriate scale empirically/experimentally (manually). Now we process the image at multiple scales. For example, we start with a

resolution of 1 and each time decrease it by 10 % or set the scaling factor to 1.2 or 1.5. The pyramid construction method should be chosen empirically. At each scale, we implement the detection.

**R** It is important to realise that it is possible to build a pyramid in which the resolution of the input image is increased. This allows the detection of small silhouettes (although the effectiveness of this approach is not very high).

**R** For testing, please choose a relatively small image with two silhouettes of different sizes. For example `testImage4.png`. However, please do not expect completely error-free operation of the system. Primarily because the set of negative samples was not very large.

7. Finally, the issue of visualisation and possibly integration of detections needs to be resolved. It is best to memorise the coordinates of the windows where silhouettes have been detected, scale them to base resolution and plot them.
8. In addition, a non-maximum suppression (NMS) method is used to eliminate multiple detections of the same silhouette.

